PROJECT PROGRESS REPORT: FOCUSLOCK

A Hardware-Free Webcam-Based Gaze-Control System

Project Title: FocusLock (Gaze-Controlled Presentation System)	**Date:** Academic Term Progress
Domain: Computer Vision, HCI, Signal Processing	**Tech Stack:** Python, OpenCV, MediaPipe, NumPy
Current Status: Stages 1, 2, 3 Implemented & Verified (Working MVP)	**Target Delivery:** 6-Stage Working Demonstration

1. Project Objective & Methodology

The objective of **FocusLock** is to develop a contact-free, low-cost presentation navigation system using only a standard laptop webcam. Commercial eye-trackers depend on expensive dedicated infrared cameras. FocusLock achieves real-time gaze tracking through a structured 6-stage software pipeline.

Core Design Methodology: MediaPipe is utilized exclusively as the low-level geometric sensor. The core intellectual and algorithmic contributions of this project are the subsequent stages developed from scratch: **Spatial Normalization**, **Moving-Average Jitter Filtering**, **5-Point Linear Calibration**, and **Dwell-Based Intent Recognition**.

2. Implementation Status Summary

Stage	Module Name	Status	Deliverable Accomplished
Stage 1	Sensing & Iris Extraction	COMPLETED	MediaPipe 478-landmark tracking, iris centroid extraction (468/473).
Stage 2	Gaze Normalization & Filter	COMPLETED	Eye-box ratio calculation, dual-eye fusion, sliding-window jitter filter.
Stage 3	5-Point Screen Calibration	COMPLETED	Interactive 5-target routine, 1D least-squares polynomial fit, JSON caching.
Stage 4	Dwell Intent Detection	IN PROGRESS	Continuous 1.8s dwell timer logic to eliminate the 'Midas Touch' error.
Stage 5	Presentation Controller	NEXT PHASE	PyAutoGUI slide-advance keyboard binding with safe DEMO_MODE toggle.
Stage 6	Telemetry Logging & Plots	NEXT PHASE	CSV real-time coordinate logger for empirical latency & precision graphs.

3. Detailed Work Completed (Stages 1 – 3)

Stage 1: MediaPipe Iris & Landmark Extraction (tracking/iris_tracker.py)

- **Modern MediaPipe Tasks API:** Configured FaceLandmarker with the standalone 478-point face_landmarker.task model bundle, resolving Python 3.13 compatibility.
- **Iris Coordinates:** Extracted pixel centroids for right iris (Index 468) and left iris (Index 473).
- **Real-Time Visual Feedback:** Integrated dynamic HUD overlay rendering live FPS, face detection confidence, and pupil indicators.

Stage 2: Gaze Normalization & Moving-Average Filter (tracking/gaze_estimator.py, filtering/moving_average.py)

- **Mathematical Normalization:** Formulated scale-invariant ratios within the eye bounding box: Gaze_X = (Iris_X - X_min) / (X_max - X_min). This ensures head movements do not distort eye-relative measurements.
- **Dual-Eye Fusion:** Averaged left and right eye vectors to mitigate unilateral eyelid occlusion.

- **Jitter Reduction Filter:** Implemented a FIFO sliding-window moving average filter (Window Size N=7). This successfully eliminates high-frequency sensor tremor and microsaccades with minimal perceptual latency (~230ms).
- **Visual Gaze Radar:** Created a real-time 2D radar HUD visually comparing RAW (Red) vs FILTERED (Green) gaze trajectories.

Stage 3: 5-Point Calibration & Screen Coordinate Mapping (calibration/calibrator.py, ui/overlay.py)

- **Why Calibration Was Essential:** Physiological eye rotation across a monitor only spans a narrow band (0.35 to 0.65) in normalized space. Calibration amplifies and shifts this range to cover the full screen [0.0, 1.0].
- **5-Target Routine:** Built an interactive state machine displaying Top-Left, Top-Right, Center, Bottom-Left, and Bottom-Right visual targets, collecting 30 sample frames per point.
- **Linear Regression Mapping:** Computed optimal gain (m) and offset (c) per axis using numpy.polyfit: X_screen = m_x * Gaze_X + c_x.
- **Persistence:** Calibration coefficients and target centroids are saved in calibration.json for instant re-use.
- **Interactive Overlay:** Rendered calibrated crosshairs and defined left (0-15%) and right (85-100%) trigger zones on the live camera viewport.

4. Key Technical Challenge & Solution

Challenge: In Python 3.13 and MediaPipe 0.10.x, the legacy mediapipe.solutions API was completely deprecated, causing runtime import crashes.
Solution: Re-engineered the tracking subsystem to interface with the modern **MediaPipe Tasks Python API** (mediapipe.tasks.python.vision.FaceLandmarker), including automated model asset caching and explicit landmark connection rendering.

5. Next Immediate Steps (To Complete MVP)

1. **Stage 4 (Dwell Intent Detection):** Implement continuous timer thresholding (1.8s) when gaze enters trigger zones to prevent accidental activations.
2. **Stage 5 (Presentation Actuation):** Integrate pyautogui.press('right') / pyautogui.press('left') with a safe DEMO_MODE toggle.
3. **Stage 6 (Telemetry Logging & Benchmark Report):** Log real-time frame telemetry to CSV and plot quantitative accuracy curves for the final viva presentation.